

K-Means Clustering

K-Means Clustering হলো একটি **unsupervised learning** algorithm — এখন পর্যন্ত যেগুলো শিখেছে (Decision Tree, KNN, SVM) সব ছিল **supervised** (মানে label/answer আগে থেকে জানা ছিল)। কিন্তু K-Means-এ কোনো label থাকে না — এটা নিজে থেকেই data-কে গ্রুপে ভাগ করে।

K-Means আসলে কী করে

ধরো, তোমার কাছে অনেক customer-এর data আছে (Age, Salary) কিন্তু তুমি জানো না কে কোন group-এর। K-Means এই customer-দের **similarity অনুযায়ী গ্রুপ (cluster)-এ ভাগ করে দেয়** — একই রকম বৈশিষ্ট্যের customer-রা একই cluster-এ পড়ে।

সহজ Example: একটি ক্লাসে ৫০ জন ছাত্র আছে, কিন্তু কোনো section (A, B, C) ঠিক করা নেই। তুমি যদি তাদের height, weight দেখে similar ছাত্রদের একসাথে গ্রুপ করো — এটাই clustering।

"K" মানে কী

K = কতগুলো cluster (গ্রুপ) তুমি বানাতে চাও — এটা তোমাকে আগে থেকেই বলে দিতে হয়। যেমন K=3 মানে ৩টা group-এ data ভাগ করতে হবে।

কীভাবে কাজ করে (Step by Step)

Step 1: K সংখ্যক Random Centroid বসাও

প্রথমে randomly K টা point বেছে নেওয়া হয় — এগুলোকে বলে **centroid** (প্রতিটা cluster-এর কেন্দ্রবিন্দু)।

Step 2: প্রতিটা Data Point-কে কাছের Centroid-এ Assign করো

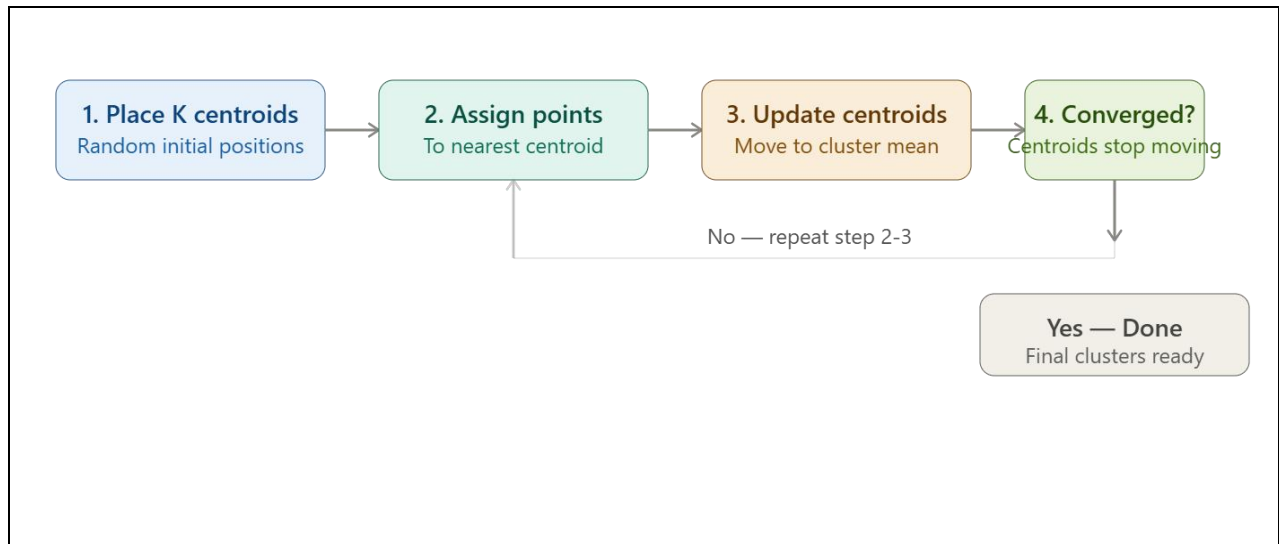
প্রতিটা data point থেকে সবগুলো centroid-এর distance measure করা হয় (সাধারণত Euclidean distance), আর যে centroid-টা সবচেয়ে কাছে, সেই cluster-এ point-টাকে দেওয়া হয়।

Step 3: Centroid Update করো

প্রতিটা cluster-এ যে data point গুলো জমা হলো, তাদের **average (mean)** বের করে নতুন centroid বসানো হয় — এই কারণেই নাম "K-Means" (mean বের করে centroid সরানো হয়)।

Step 4: Repeat করো (Step 2-3 বারবার)

Centroid নতুন জায়গায় গেলে, আবার প্রতিটা point-কে নতুন করে কাছের centroid-এ assign করা হয়, আবার centroid update হয় — এটা চলতে থাকে যতক্ষণ না centroid গুলো আর নড়ে (convergence)।



Best K কীভাবে বেছে নেবে — Elbow Method

সমস্যা হলো — তুমি আগে থেকে কীভাবে জানবে K কত হওয়া উচিত (৩ নাকি ৫ নাকি ১০)? এর জন্য ব্যবহার হয় Elbow Method।

কীভাবে কাজ করে

- প্রতিটা K value-এর জন্য (K=1,2,3...10) K-Means চালিয়ে WCSS (Within-Cluster Sum of Squares) বের করা হয় — মানে প্রতিটা point তার cluster-এর centroid থেকে কতটা দূরে আছে তার sum।
- K বাড়ালে WCSS কমতে থাকে (স্বাভাবিক, বেশি cluster মানে প্রতিটা group ছোট হয়ে যায়)
- একটা graph plot করা হয় (K vs WCSS), আর যেখানে graph-এ হাতের কনুই (elbow)-এর মতো বাঁক দেখা যায়, সেই K-টাই best choice — কারণ এর পরে K বাড়ালে WCSS আর তেমন কমে না।

Scikit-learn-এ Code

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Distance-based algorithm, তাই scaling জরুরি
scaler = StandardScaler()
```

```

X_scaled = scaler.fit_transform(X)

# Elbow method দিয়ে best K বের করা
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('K')
plt.ylabel('WCSS')
plt.show()

# Best K (ধরো elbow point = 3) দিয়ে final model
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

```

গুরুত্বপূর্ণ কথা — এটা Supervised না

K-Means-এ কোনো "সঠিক উত্তর" (label) দেওয়া হয় না। Algorithm নিজেই বের করে কোন data একসাথে similar — তাই একে বলে **unsupervised learning**।

K-Means-এর সুবিধা

- সহজ ও দ্রুত — বড় dataset-এও ভালো কাজ করে
- বোঝা এবং implement করা সহজ
- অনেক ক্ষেত্রে ব্যবহার হয় — customer segmentation, image compression, document grouping

K-Means-এর অসুবিধা

- **K আগে থেকে বলে দিতে হয়** — automatically বের করতে পারে না
- **Outlier-এর প্রতি sensitive** — একটা extreme value centroid-কে ভুল জায়গায় টেনে নিতে পারে
- **Random initial centroid** এর উপর নির্ভরশীল — ভিন্ন starting point-এ ভিন্ন result আসতে পারে (তাই `n_init` parameter দিয়ে একাধিকবার try করানো হয়)
- Cluster গুলো round/spherical shape ধরে নেয় — যদি data-এর shape জটিল হয় (যেমন crescent shape), তাহলে ভালো কাজ করে না

এক লাইনে সারমর্ম: K-Means হলো এমন একটা algorithm যেটা কোনো label ছাড়াই, শুধু similarity (distance) দেখে data-কে K টা group-এ ভাগ করে দেয়, আর প্রতিটা group-এর কেন্দ্রবিন্দু (centroid) বারবার update করে যতক্ষণ না সেটা স্থির হয়ে যায়।

চলো Titanic dataset দিয়ে actual K-Means apply করে দেখাই — step by step।

Step 1: Data Load ও Prepare করা

Titanic dataset-এ clustering-এর জন্য আমরা numerical feature গুলো ব্যবহার করবো — Age আর Fare (কারণ K-Means শুধু numerical data নিয়ে কাজ করে, distance calculate করতে হয়)।

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

df = pd.read_csv('titanic.csv')

# শুধু numerical feature বেছে নেওয়া
X = df[['Age', 'Fare']].copy()

# Missing value handle করা (আগে যেমন শিখেছিলে)
X['Age'] = X['Age'].fillna(X['Age'].median())
X['Fare'] = X['Fare'].fillna(X['Fare'].median())
```

Step 2: Scaling করা (Must!)

Age-এর range (0-80) আর Fare-এর range (0-500+) অনেক আলাদা — scaling না করলে Fare distance calculation-এ dominate করে ফেলবে।

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 3: Elbow Method দিয়ে Best K বের করা

```
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method for Titanic (Age vs Fare)')
```

```
plt.show()
```

Titanic-এর এই dataset-এ সাধারণত **K=3 বা K=4** এর কাছাকাছি একটা elbow দেখা যায় (Fare-এ কিছু extreme value থাকার কারণে group গুলো clear ভাবে আলাদা হয়ে যায়)।

Step 4: Final K-Means Apply করা

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)

print(df[['Age', 'Fare', 'Cluster']].head(10))
```

Step 5: Cluster গুলো Visualize করা

```
plt.figure(figsize=(8, 6))
plt.scatter(df['Age'], df['Fare'], c=df['Cluster'], cmap='viridis', alpha=0.6)
plt.xlabel('Age')
plt.ylabel('Fare')
plt.title("Titanic Passengers Clustered by Age & Fare")
plt.colorbar(label='Cluster')
plt.show()
```

Result কী দেখাবে (Interpretation)

Titanic-এ Age আর Fare দিয়ে cluster করলে সাধারণত এমন group পাওয়া যায়:

Cluster	বৈশিষ্ট্য
Cluster 0	কম Fare, মাঝবয়সী passenger — সম্ভবত 3rd class
Cluster 1	বেশি Fare, বিভিন্ন বয়স — সম্ভবত 1st class
Cluster 2	মাঝারি Fare, নির্দিষ্ট বয়সসীমা — সম্ভবত 2nd class

মজার ব্যাপার: আমরা কোথাও Pclass (passenger class) ব্যবহার করিনি, কিন্তু K-Means শুধু Age আর Fare দেখেই প্রায় Pclass-এর মতো একটা grouping বের করে ফেলতে পারে — কারণ Fare আর Pclass-এর মধ্যে স্বাভাবিক সম্পর্ক আছে (1st class-এর টিকিট বেশি দামি)।

Bonus: Cluster-এর গড় (Mean) দেখে বোঝা

```
print(df.groupby('Cluster')[['Age', 'Fare']].mean())
```

এতে প্রতিটা cluster-এর average Age আর Fare দেখে বুঝতে পারবে প্রতিটা group আসলে কী represent করছে।